

Simulated Annealing Demonstrative Example

SULAGNA ROY

2026-09-12

Simulated Annealing

Demonstrative example

This R code demonstrates Simulated Annealing for finding the value x that maximizes

$$f(x) = (\cos(50x) + \sin(20x))^2, 0 < x < 1.$$

Step 1 : Objective Function

```
set.seed(1)
fn <- function(x, T = 1){
  h <- ( cos(50*x) + sin(20*x) )^2 # The actual function being optimized
  exp(h/T) * (0 < x & x < 1) # ensures that values outside (0,1) receive 0
  # but the algorithm uses exp(h/T) as the target/acceptance function.
}
```

Step 2 : Plot of Objective Function

```
x <- seq(0, 1, length = 5e2)
plot(x, fn(x), type = 'l', ylim = c(0,250), ylab = "exp(f/T)", lwd=1.5)

# Effect of Temperature
tseq = c(.83, .75, .71)

# The temperature T controls how strongly the algorithm concentrates around high values of h(x).

# High T implies flatter distribution implying more exploration
# Low T implies sharper peaks implying more exploitation

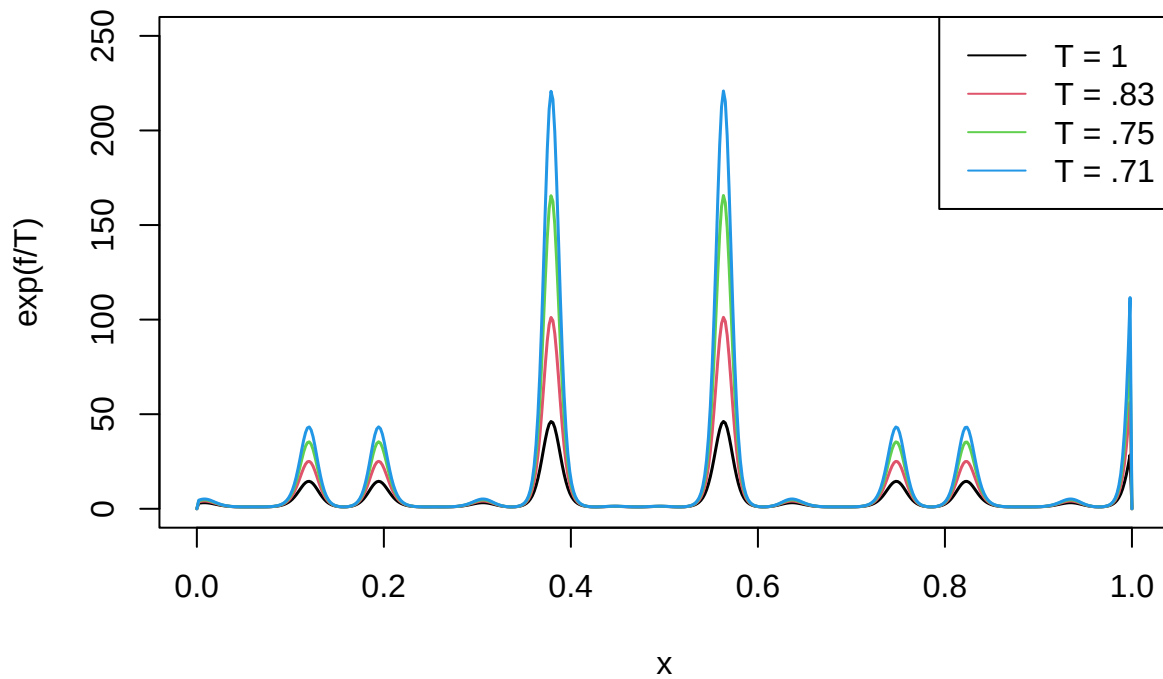
# So as T changes from 0.83 to 0.75 to 0.71 the peaks become increasingly pronounced.

for(t in 1:3){
  lines(x, fn(x, T = tseq[t]), col = t+1, lwd=1.5)
```

```

}
legend("topright", col = 1:4, lty = 1,
      legend = c("T = 1", "T = .83", "T = .75", "T = .71"))

```



Step 3: Simulated Annealing Algorithm

```

# r=.3 controls the size of the possible movement
simAn <- function(N = 10, r = .3){
  x <- numeric(length = N)

  # Initial Value
  x[1] <- runif(1)

  for(k in 2:N){

    # Generate a candidate
    a <- runif(1, x[k-1] - r, x[k-1] + r)

    # Decrease Temperature this is the cooling schedule
    T <- 1/(log(k))

    # Calculate Acceptance Ratio

```

```

ratio <- fn(a,T)/fn(x[k-1], T)  #fn is the exp(h/T)
if( runif(1) < ratio)
{
  # If the ratio is greater than 1 candidate is automatically accepted
  x[k] <- a
}else
{
  x[k] <- x[k-1]
  # If the candidate is worse, the ratio is less than 1,
  # but it can still be accepted with some probability.
  # This is the key idea of simulated annealing:
  # occasionally accepting a worse solution helps avoid getting trapped in a local maximum.
}
}
return(x)
}

```

Step 4: Find the estimated Optimizer

```

N <- 500
sim <- simAn(N = N)

# This finds the simulated value having the largest value of fn()
sim[which.max(fn(sim))]  # theta^*

```

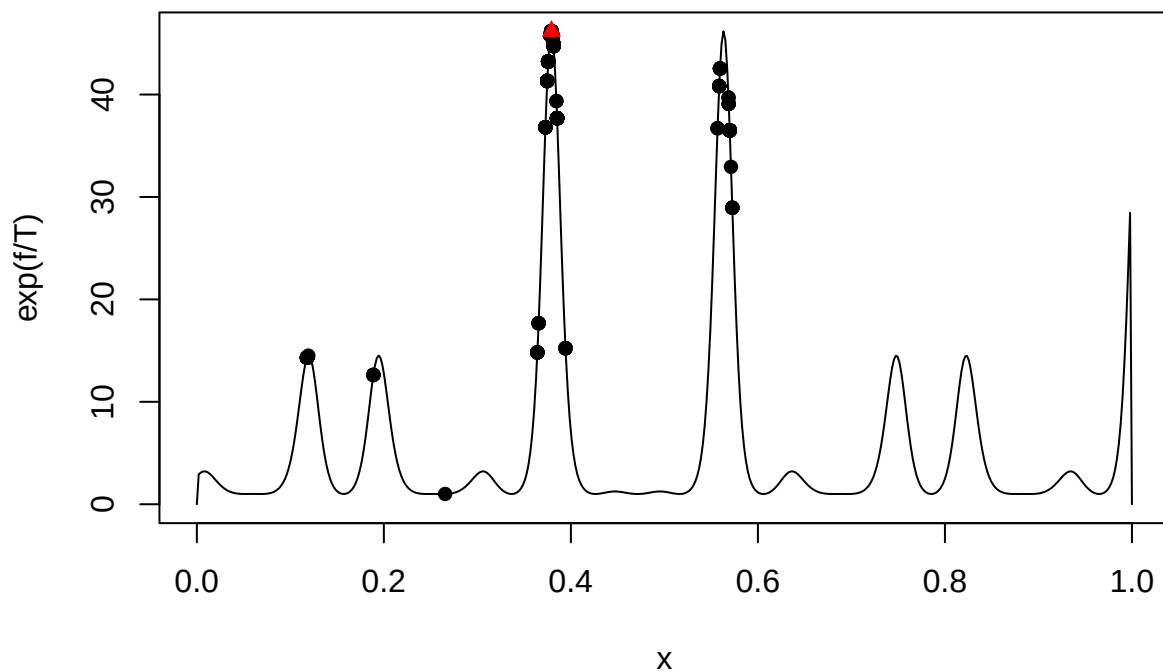
```
## [1] 0.3792136
```

Step 5: Final Plot

```

plot(x, fn(x), type = 'l', ylab = "exp(f/T)")
points(sim, fn(sim), pch = 16, col = 1)
points(sim[which.max(fn(sim))],fn(sim[which.max(fn(sim))]),pch=17,col="red")

```



Step 6: Comment

Simulated annealing starts by exploring the whole parameter space and gradually becomes more selective as the temperature decreases, allowing it to escape local maxima and approach the global maximum.
